

FAG 非确定性计算严格按列 Swizzle 分核方案 (Dense / Causal)

版本: v1.2
日期: 2026-08-13
范围: arch35、非 TND、Dense/Causal (包含 GQA/MQA)、非确定性计算
参考: 确定性 CalDenseSwizzleIndex、CalCausalSwizzleIndex 和 NoGQA/casual.py

1. 修订结论

v1.0 只是将任务按 (batch,s2,s1) 列优先展开，再按固定连续任务块分核。当列长大于连续块时，同一实际列会被拆给多个核，因此不满足严格“按列分核”。

v1.1 将分核基本单元改为完整列，并把以下条件作为正确性硬约束：

```
对于任意实际列 (batch, s2_idx):
owner(batch, s2_idx) 唯一;
该列全部有效 s1 块均由 owner 处理;
其他核不得处理该列中的任何块。
```

修改后的方案：

- Dense: 实际列 (batch,s2) 轮转分配给核，核内完整遍历该列的 m 行；
- Causal 偶数部分: 保留确定性方案的“双 batch 三角拼矩形”，但以完整虚拟列分核；
- Causal 奇数尾 batch: 实际三角列轮转分核，各核把自己拥有的变长列紧凑拼接；
- 删除 continuousBlockNum 对列内任务的切割，避免一列跨核；
- 不保留确定性“同轮同行互斥”限制，允许同一 local_step 出现同行任务。

v1.2 不再限制 $g == 1$ 。算子将 (bo,n2,g) 展平为逻辑 batch，Dense/Causal 轮次与坐标公式直接处理 $B=b*n2*g$ 。

2. 符号和坐标

Python 参考实现统一使用 0-based 坐标。

符号	含义
k	参与分核的 AIC 数
m	s1Outer, 基本块行数
n	s2Outer, 基本块列数
B	展平逻辑 batch 数, 即 $b*n2*g$
j	core_id, 范围 [0,k)
r	当前核的 local_step
(w,x,y)	(batch,s1_idx,s2_idx)

对于 $g > 1$, w 唯一解码为 $(bo, n2o, go)$, 严格列 owner 的逻辑列定义为 $(bo, n2o, go, s2_idx)$ 。不同 go 可以共享同一 K/V 物理列, 非确定性原子累加路径允许该物理列由多核处理。

3. Dense 严格按列分核

3.1 列 owner

Dense 共有 $B*n$ 个实际列, 按 $(batch, s2)$ 展平:

```
columnId = batch * n + s2_idx
owner    = columnId % k
```

因此每个 $columnId$ 只有一个 owner。

3.2 核内坐标

每个核依次领取:

```
columnId = j, j+k, j+2k, ...
```

每领取一列, 连续处理该列全部 m 行:

```
localColumn = r // m
x           = r % m
columnId    = localColumn * k + j

if columnId >= B*n:
    return invalid

w, y = divmod(columnId, n)
return (w, x, y)
```

统一循环上界:

```
R_dense = ceil(B*n / k) * m
```

3.3 与确定性 Dense 的区别

确定性 Dense 会对行号做循环移位, 保证同一轮中一个 $(batch, x)$ 不被多个核同时命中。非确定性路径不需要该限制, 因此直接使用 $x=r\%m$:

- 保留完整列归核和右矩阵复用;
- 删除行循环移位;
- 允许多个核在同一 $local_step$ 处理同一 $batch$ 的同一行、不同列。

严格整列分配后, 最小负载粒度从任务块变为一列, Dense 每核负载差上界为 m 。

4. Causal 严格按列分核

4.1 首版范围

支持块级方阵下三角：

```
m == n
0 <= y <= x < m
```

非等长 Right-Down Causal 和 TND 暂时回退原路径；GQA/MQA 通过 $B=b*n2*g$ 展平直接支持。

4.2 两个 batch 拼成矩形

单个下三角任务数是 $m*(m+1)/2$ ，两个 batch 正好拼成无 padding 的 $m*(m+1)$ 虚拟矩形。

一个 batch pair 有 $m+1$ 个虚拟列，每个虚拟列固定包含 m 个任务。所有配对区虚拟列总数：

```
pairCount = B // 2
pairColumns = pairCount * (m + 1)
```

虚拟列严格轮转归核：

```
virtualColumnId = localColumn * k + j
virtualX = r % m
```

对：

```
pairId, virtualY = divmod(virtualColumnId, m+1)
```

执行坐标还原：

```
if virtualY <= virtualX:
    w = 2*pairId
    x = virtualX
    y = virtualY
else:
    w = 2*pairId + 1
    x = m - 1 - virtualX
    y = m - virtualY
```

4.3 为什么实际列不会被拆核

对第一个 batch：

```
实际列 y 只出现在 virtualY=y
```

对第二个 batch:

实际列 y 只出现在 $\text{virtualY} = m - y$

而一个 virtualY 只属于一个 virtualColumnId ，一个虚拟列又只属于一个核，因此两个 batch 的每个实际列也都只有一个 owner。

实际列 owner 可直接计算：

```
pairId = batch // 2
virtualY = y          # batch 为偶数
virtualY = m - y      # batch 为奇数
owner = (pairId*(m+1) + virtualY) % k
```

一个虚拟列可能先处理第二个 batch 的一列，再处理第一个 batch 的一列，但不会把任何实际列拆开。

4.4 奇数 batch 尾三角

当 B 为奇数时，最后一个 batch 的每个实际列完整轮转分核。为平衡配对区尾轮，最长列从配对区少领取一个虚拟列的核开始分配：

```
start = pairColumns % k
owner(tailBatch, y) = (y + start) % k
```

对核 j :

```
residue = (j - start + k) % k
y = residue, residue+k, residue+2k, ...
```

列 y 的长度为 $m - y$ 。核内将自己拥有的列紧凑拼接，不使用固定 m padding。

前 q 个尾列的任务前缀：

```
prefix(q) = q*(m-residue) - k*q*(q-1)/2
```

通过整数平方根反解 $\text{prefix}(q) \leq \text{tailTask} < \text{prefix}(q+1)$ ，再得到：

```
y = residue + q*k
x = y + tailTask - prefix(q)
```

Python 使用整数平方根和精确边界修正；迁移到 Kernel 时可复用现有平方根反解方式。

4.5 负载上界

配对区每个虚拟列长度均为 m ，轮转分配的核间差最多一列。奇数尾段通过 $\text{start} = \text{pairColumns} \% k$ 将长列优先交给配对区少拿一列的核。

构造后的总负载满足：

$$\max(\text{coreLoad}) - \min(\text{coreLoad}) \leq m$$

该条件已对 $k=1..64$ 、 $m=1..256$ 、 $B=1..9$ 做解析公式穷举，并在逐坐标测试中同步检查。

5. Host/Kernel 实现约束

本方案已迁移到 ops-transformer-xkbug/attention/flash_attention_score_grad 的 arch35 Kernel 实现。

5.1 Tiling 条件

不新增 schedule mode，不改变 isSplitByBlockIdx 的布尔含义，Kernel 继续使用现有 enableDenseSwizzle 和 enableCasualSwizzle 作为唯一入口。

```
common:
  !isDeterministic
  layout != TND
  splitAxis == BN2GS1S2
  enableSwizzle
  isSplitByBlockIdx
  k > 0, m > 0, n > 0, B > 0

dense:
  sparseType == DENSE

causal:
  sparseType == CASUAL
  s1Outer == s2Outer
```

性能启用门槛应通过 NPU 实测确定。严格按列后，如果总列数远小于 k ，有效核数不足，建议回退原分核。

5.2 Kernel 接口

建议新增或替换为以下坐标函数：

```
CalNonDeterDenseColumnIndex(coreId, localStep)
CalNonDeterCausalColumnIndex(coreId, localStep)
```

随后统一填充：

```
flatBatch = (boldx*n2 + n2oldx)*g + goldx
boldx     = flatBatch // (n2*g)
batchTail = flatBatch % (n2*g)
n2oldx    = batchTail // g
goldx     = batchTail % g
s1oldx    = x
```

```
s2oldx  = y
s2CvBegin / s2CvEnd
```

Kernel 初始化和坐标分发必须读取同一个 enableDenseSwizzle / enableCasualSwizzle 分支。swizzleMaxRound 与 (w,x,y) 坐标均使用本文同一套整列公式。

5.3 回退范围

- TND;
- 非方阵 Causal、非等长 Right-Down;
- BAND/PREFIX 等其他稀疏模式;
- 总列数过少、整列分核并行度不足;
- mode 非法或 tiling 数据不完整。

6. 验证方法

每组参数检查:

1. 坐标不越界;
2. 每个有效任务恰好出现一次;
3. Dense 覆盖数为 $B*m*n$;
4. Causal 覆盖数为 $B*m*(m+1)/2$ 且全部满足 $y \leq x$;
5. 每个 (batch,y) 的 owner 集合大小严格等于 1;
6. 由显式 column_owner() 反算的 owner 与生成坐标的 core 一致;
7. 每核负载差不超过 m;
8. 循环上界之后所有核均不再返回有效任务;
9. Dense/Causal 均能观察到允许的同 local-step 同行任务;
10. 大整数尾三角反解保持正确。

7. 本版验证结果

执行:

```
python .\non_deter_swizzle.py --full-test --demo
```

结果:

```
PASS dense: cases=4968, coordinates=6574396, columns=172735,
    max_load_skew/m=1.00 <= 1.00
PASS causal: cases=920, coordinates=4065476, columns=107593,
    max_load_skew/m=1.00 <= 1.00
PASS strict column ownership: every (batch, s2_idx) has exactly one owner core
PASS relaxed constraint: same-row tasks in one local_step are allowed and observed
dense sample same-column transition=95.74%
causal sample same-column transition=90.00%
```

合计检查:

- 5,888 组逐坐标配置;
- 10,639,872 个有效坐标;
- 280,328 个实际列;
- 核数覆盖 1..8、16、20、24、32、48;
- 随机 shape 最大到 128×128 block;
- Causal 奇数/偶数 batch 均覆盖;
- Causal 解析负载额外穷举 $k=1..64$ 、 $m=1..256$ 、 $B=1..9$;
- 尾三角整数反解抽查到 $m=1,000,000$ 。
- GQA/MQA 展平坐标额外检查 48 组配置、44,104 个坐标, 覆盖 $g=2/4/8$ 。

所有实际列均为单 owner, 没有发现跨核拆列。

8. Python 参考实现

文件: non_deter_swizzle.py

关键接口:

```
dense_position(k, m, n, b, core_id, local_step)
dense_column_owner(k, n, b, batch, s2_idx)
dense_max_local_steps(k, m, n, b)

causal_position(k, m, b, core_id, local_step)
causal_column_owner(k, m, b, batch, s2_idx)
causal_max_local_steps(k, m, b)

run_self_test(full=False)
```

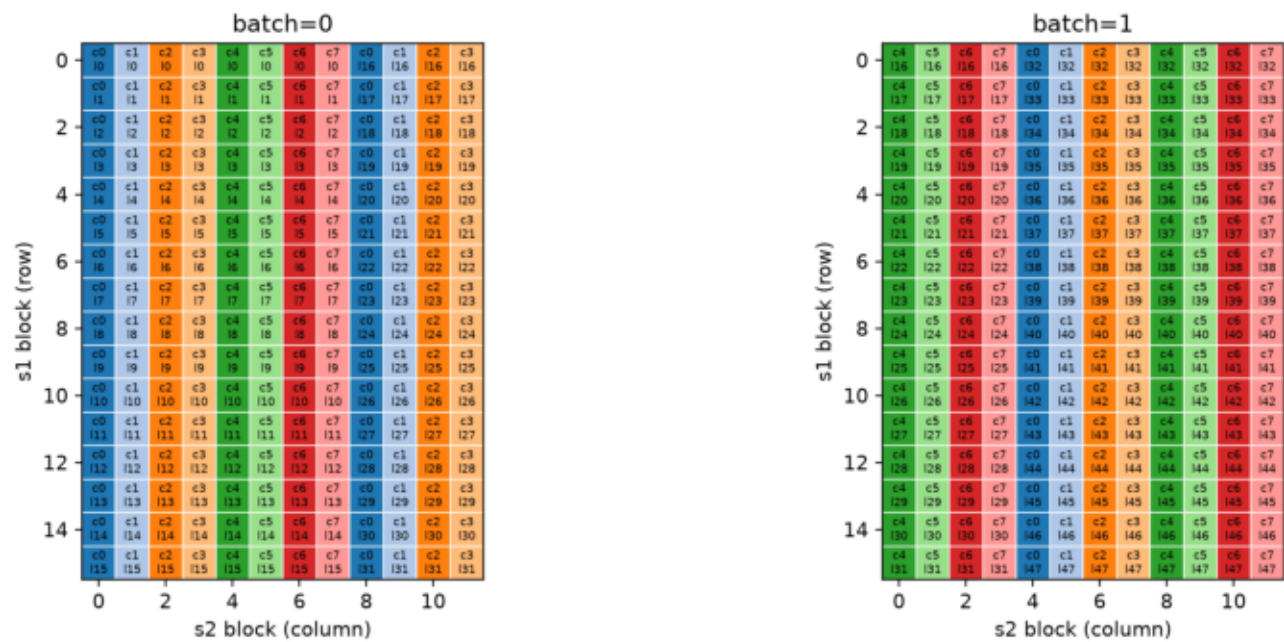
参考实现不依赖第三方库即可完成验证; 生成示意图时需要 numpy/matplotlib。

9. 风险和后续验证

- 严格按列后最小调度粒度是一整列, 列数少于核数时并行度下降; Host 需要性能门槛。
- Python 只验证任务和坐标性质, 不能替代 Kernel 的原子累加、workspace 汇聚和流水同步检查。
- C++ 实现需持续保持轮次初始化与 Kernel 坐标分发一致, 并回归每核 owner 和 (w,x,y) 。
- GQA/MQA 中的唯一 owner 指逻辑列 $(bo,n2o,go,s2_idx)$; 若后续需要同一 K/V 物理列也严格单 owner, 需另行设计跨 g 的联合分核。
- 仍需执行 CPU/NPU 精度、确定性开关回归和性能测试, 才能决定默认启用范围。

Dense: 严格按实际列轮转分核示意

Strict column-owned non-deterministic DENSE swizzle | k=8, m=16, n=12, b=2, same-column transition=95.7%



Causal: 双 Batch 拼接与奇数尾三角分核示意

Strict column-owned non-deterministic CAUSAL swizzle | $k=8$, $m=16$, $n=16$, $b=3$, same-column transition=90.0%

